

DistributedDataParallel#

<https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html>

Description:

Implement distributed data parallelism based on `torch.distributed` at module level. This container provides data parallelism by synchronizing gradients across each model replica. The devices to synchronize across are specified by the input `process_group`, which is the entire world by default. Note that `DistributedDataParallel` does not chunk or otherwise shard the input across participating GPUs; the user is responsible for defining how to do so, for example through the use of a `DistributedSampler`. See also: Basics and Use `nn.parallel.DistributedDataParallel` instead of `multiprocessing` or `nn.DataParallel`. The same constraints on input as in `torch.nn.DataParallel` apply. Creation of this class requires that `torch.distributed` to be already initialized, by calling `torch.distributed.init_process_group()`. `DistributedDataParallel` is proven to be significantly faster than `torch.nn.DataParallel` for single-node multi-GPU data parallel training.

Function Signature

```
class torch.nn.parallel.DistributedDataParallel(module,  
device_ids=None, output_device=None, dim=0,  
broadcast_buffers=True, init_sync=True, process_group=None,  
bucket_cap_mb=None, find_unused_parameters=False,  
check_reduction=False, gradient_as_bucket_view=False,  
static_graph=False, delay_all_reduce_named_params=None,  
param_to_hook_all_reduce=None, mixed_precision=None,  
device_mesh=None, skip_all_reduce_unused_params=False)  
[source]#
```

Parameters

Example::

Variables

```
join(divide_by_initial_world_size=True, enable=True,  
throw_on_early_termination=False)[source]#
```

Code Examples

```
>>> torch.cuda.set_device(i)
```

```
>>> torch.accelerator.set_device_index(i)
```

```
>>> if torch.accelerator.is_available():
>>>     device_type =
torch.accelerator.current_accelerator().type
>>>     vendor_backend =
torch.distributed.get_default_backend_for_device(device_type)
>>>
>>> torch.distributed.init_process_group(
>>>     backend=vendor_backend, world_size=N, init_method='...'
>>> )
>>> model = DistributedDataParallel(model, device_ids=[i],
output_device=i)
```

```
>>> torch.distributed.init_process_group(device_id=i)
```

```
>>> import torch.distributed.autograd as dist_autograd
>>> from torch.nn.parallel import DistributedDataParallel as DDP
>>> import torch
>>> from torch import optim
>>> from torch.distributed.optim import DistributedOptimizer
>>> import torch.distributed.rpc as rpc
>>> from torch.distributed.rpc import RRef
>>>
>>> t1 = torch.rand((3, 3), requires_grad=True)
>>> t2 = torch.rand((3, 3), requires_grad=True)
>>> rref = rpc.remote("worker1", torch.add, args=(t1, t2))
>>> ddp_model = DDP(my_model)
>>>
>>> # Setup optimizer
>>> optimizer_params = [rref]
>>> for param in ddp_model.parameters():
>>>     optimizer_params.append(RRef(param))
>>>
>>> dist_optim = DistributedOptimizer(
>>>     optim.SGD,
>>>     optimizer_params,
>>>     lr=0.05,
>>> )
>>>
>>> with dist_autograd.context() as context_id:
>>>     pred = ddp_model(rref.to_here())
>>>     loss = loss_func(pred, target)
>>>     dist_autograd.backward(context_id, [loss])
>>>     dist_optim.step(context_id)
```

```
>>> model_DDP = torch.nn.parallel.DistributedDataParallel(model)
>>> # Training loop
>>> ...
>>> ddp_logging_data = model_DDP._get_ddp_logging_data()
>>> static_graph = ddp_logging_data.get("can_set_static_graph")
```

```
>>> torch.distributed.init_process_group(backend='nccl',
world_size=4, init_method='...')
>>> net = torch.nn.parallel.DistributedDataParallel(model)
```

```

>>> import torch
>>> import torch.distributed as dist
>>> import os
>>> import torch.multiprocessing as mp
>>> import torch.nn as nn
>>> # On each spawned worker
>>> def worker(rank):
>>>     dist.init_process_group("nccl", rank=rank, world_size=2)
>>>     torch.cuda.set_device(rank)
>>>     model = nn.Linear(1, 1, bias=False).to(rank)
>>>     model = torch.nn.parallel.DistributedDataParallel(
>>>         model, device_ids=[rank], output_device=rank
>>>     )
>>>     # Rank 1 gets one more input than rank 0.
>>>     inputs = [torch.tensor([1]).float() for _ in range(10 +
rank)]
>>>     with model.join():
>>>         for _ in range(5):
>>>             for inp in inputs:
>>>                 loss = model(inp).sum()
>>>                 loss.backward()
>>>     # Without the join() API, the below synchronization will
hang
>>>     # blocking for rank 1's allreduce to complete.
>>>     torch.cuda.synchronize(device=rank)

```

```

>>> ddp = torch.nn.parallel.DistributedDataParallel(model, pg)
>>> with ddp.no_sync():
>>>     for input in inputs:
>>>         ddp(input).backward() # no synchronization,
accumulate grads
>>> ddp(another_input).backward() # synchronize grads

```

```
>>> def noop(state: object, bucket: dist.GradBucket) ->
torch.futures.Future[torch.Tensor]:
>>>     fut = torch.futures.Future()
>>>     fut.set_result(bucket.buffer())
>>>     return fut
>>> ddp.register_comm_hook(state=None, hook=noop)
```

```
>>> def encode_and_decode(state: object, bucket:
dist.GradBucket) -> torch.futures.Future[torch.Tensor]:
>>>     encoded_tensor = encode(bucket.buffer()) # encode
gradients
>>>     fut =
torch.distributed.all_reduce(encoded_tensor).get_future()
>>>     # Define the then callback to decode.
>>>     def decode(fut):
>>>         decoded_tensor = decode(fut.value()[0]) # decode
gradients
>>>         return decoded_tensor
>>>     return fut.then(decode)
>>> ddp.register_comm_hook(state=None, hook=encode_and_decode)
```

Notes & Warnings

NOTE Please refer to PyTorch Distributed Overview for a brief introduction to all features related to distributed training.

NOTE

Note `DistributedDataParallel` can be used in conjunction with `torch.distributed.optim.ZeroRedundancyOptimizer` to reduce per-rank optimizer states memory footprint. Please refer to `ZeroRedundancyOptimizer` recipe for more details.

NOTE

Note `nccl` backend is currently the fastest and highly recommended backend when using GPUs. This applies to both single-node and multi-node distributed training.

NOTE

Note This module also supports mixed-precision distributed training. This means that your model can have different types of parameters such as mixed types of `fp16` and `fp32`, the gradient reduction on these mixed types of parameters will just work fine.

NOTE

Note If you use `torch.save` on one process to checkpoint the module, and `torch.load` on some other processes to recover it, make sure that `map_location` is configured properly for every process. Without `map_location`, `torch.load` would recover the module to devices where the module was saved from.

NOTE

Note When a model is trained on M nodes with $\text{batch}=N$, the gradient will be M times smaller when compared to the same model trained on a single node with $\text{batch}=M*N$ if the loss is summed (NOT averaged as usual) across instances in a batch (because the gradients between different nodes are averaged). You should take this into consideration when you want to obtain a mathematically equivalent training process compared to the local training counterpart. But in most cases, you can just treat a `DistributedDataParallel` wrapped model, a `DataParallel` wrapped model and an ordinary model on a single GPU as the same (E.g. using the same learning rate for equivalent batch size).

NOTE

Note Parameters are never broadcast between processes. The module performs an all-reduce step on gradients and assumes that they will be modified by the optimizer in all processes in the same way. Buffers (e.g. `BatchNorm` stats) are broadcast from the module in process of rank 0, to all other replicas in the system in every iteration.

NOTE

Note If you are using DistributedDataParallel in conjunction with the Distributed RPC Framework, you should always use `torch.distributed.autograd.backward()` to compute gradients and `torch.distributed.optim.DistributedOptimizer` for optimizing parameters. Example:

```
>>> import torch.distributed.autograd as dist_autograd >>> from torch.nn.parallel import DistributedDataParallel as DDP >>> import torch >>> from torch import optim >>> from torch.distributed.optim import DistributedOptimizer >>> import torch.distributed.rpc as rpc >>> from torch.distributed.rpc import RRef >>> >>> t1 = torch.rand((3, 3), requires_grad=True) >>> t2 = torch.rand((3, 3), requires_grad=True) >>> rref = rpc.remote("worker1", torch.add, args=(t1, t2)) >>> ddp_model = DDP(my_model) >>> >>> # Setup optimizer >>> optimizer
```

NOTE

Note DistributedDataParallel currently offers limited support for gradient checkpointing with `torch.utils.checkpoint()`. If the checkpoint is done with `use_reentrant=False` (recommended), DDP will work as expected without any limitations. If, however, the checkpoint is done with `use_reentrant=True` (the default), DDP will work as expected when there are no unused parameters in the model and each layer is checkpointed at most once (make sure you are not passing `find_unused_parameters=True` to DDP). We currently do not support the case where a layer is checkpointed multiple times, or when there unused parameters in the checkpointed model.

NOTE

Note To let a non-DDP model load a state dict from a DDP model, `consume_prefix_in_state_dict_if_present()` needs to be applied to strip the prefix “module.” in the DDP state dict before loading.

⚠ WARNING

Warning Constructor, forward method, and differentiation of the output (or a function of the output of this module) are distributed synchronization points. Take that into account in case different processes might be executing different code.

⚠ WARNING

Warning This module assumes all parameters are registered in the model by the time it is created. No parameters should be added nor removed later. Same applies to buffers.

⚠ WARNING

Warning This module assumes all parameters are registered in the model of each distributed processes are in the same order. The module itself will conduct gradient allreduce following the reverse order of the registered parameters of the model. In other words, it is users’ responsibility to ensure that each distributed process has the exact same model and thus the exact same parameter registration order.

 WARNING

Warning This module allows parameters with non-rowmajor-contiguous strides. For example, your model may contain some parameters whose `torch.memory_format` is `torch.contiguous_format` and others whose format is `torch.channels_last`. However, corresponding parameters in different processes must have the same strides.

 WARNING

Warning This module doesn't work with `torch.autograd.grad()` (i.e. it will only work if gradients are to be accumulated in `.grad` attributes of parameters).

 WARNING

Warning If you plan on using this module with a nccl backend or a gloo backend (that uses Infiniband), together with a `DataLoader` that uses multiple workers, please change the multiprocessing start method to `forkserver` (Python 3 only) or `spawn`. Unfortunately Gloo (that uses Infiniband) and NCCL2 are not fork safe, and you will likely experience deadlocks if you don't change this setting.

 WARNING

Warning You should never try to change your model's parameters after wrapping up your model with `DistributedDataParallel`. Because, when wrapping up your model with `DistributedDataParallel`, the constructor of `DistributedDataParallel` will register the additional gradient reduction functions on all the parameters of the model itself at the time of construction. If you change the model's parameters afterwards, gradient reduction functions no longer match the correct set of parameters.

 WARNING

Warning Using DistributedDataParallel in conjunction with the Distributed RPC Framework is experimental and subject to change.

 WARNING

Warning If the model or training loop this context manager is wrapped around has additional distributed collective operations, such as SyncBatchNorm in the model's forward pass, then the flag `throw_on_early_termination` must be enabled. This is because this context manager is not aware of non-DDP collective communication. This flag will cause all ranks to throw when any one rank exhausts inputs, allowing these errors to be caught and recovered from across all ranks.

 WARNING

Warning The forward pass should be included inside the context manager, or else gradients will still be synchronized.

 WARNING

Warning Grad bucket's tensors will not be predivided by `world_size`. User is responsible to divide by the `world_size` in case of operations like `allreduce`.

 WARNING

Warning DDP communication hook can only be registered once and should be registered before calling backward.

⚠ WARNING

Warning The Future object that hook returns should contain a single tensor that has the same shape with the tensors inside grad bucket.

⚠ WARNING

Warning get_future API supports NCCL, and partially GLOO and MPI backends (no support for peer-to-peer operations like send/recv) and will return a torch.futures.Future.

Detailed Documentation

Documentation

Implement distributed data parallelism based on torch.distributed at module level.

This container provides data parallelism by synchronizing gradients across each model replica. The devices to synchronize across are specified by the input `process_group`, which is the entire world by default. Note that `DistributedDataParallel` does not chunk or otherwise shard the input across participating GPUs; the user is responsible for defining how to do so, for example through the use of a `DistributedSampler`.

See also: Basics and Use `nn.parallel.DistributedDataParallel` instead of multiprocessing or `nn.DataParallel`. The same constraints on input as in `torch.nn.DataParallel` apply.

Creation of this class requires that `torch.distributed` to be already initialized, by calling `torch.distributed.init_process_group()`.

`DistributedDataParallel` is proven to be significantly faster than `torch.nn.DataParallel` for

single-node multi-GPU data parallel training.

To use `DistributedDataParallel` on a host with N GPUs, you should spawn up N processes, ensuring that each process exclusively works on a single GPU from 0 to $N-1$. This can be done by either setting `CUDA_VISIBLE_DEVICES` for every process or by calling the following API for GPUs,

or calling the unified API for accelerator,

where i is from 0 to $N-1$. In each process, you should refer the following to construct this module:

Or you can use the latest API for initialization:

In order to spawn up multiple processes per node, you can use either `torch.distributed.launch` or `torch.multiprocessing.spawn`.

Please refer to [PyTorch Distributed Overview](#) for a brief introduction to all features related to distributed training.

`DistributedDataParallel` can be used in conjunction with `torch.distributed.optim.ZeroRedundancyOptimizer` to reduce per-rank optimizer states memory footprint. Please refer to [ZeroRedundancyOptimizer recipe](#) for more details.

`nccl` backend is currently the fastest and highly recommended backend when using GPUs. This applies to both single-node and multi-node distributed training.

This module also supports mixed-precision distributed training. This means that your model can have different types of parameters such as mixed types of `fp16` and `fp32`, the gradient reduction on these mixed types of parameters will just work fine.

If you use `torch.save` on one process to checkpoint the module, and `torch.load` on

some other processes to recover it, make sure that `map_location` is configured properly for every process. Without `map_location`, `torch.load` would recover the module to devices where the module was saved from.

When a model is trained on M nodes with $\text{batch} = N$, the gradient will be M times smaller when compared to the same model trained on a single node with $\text{batch} = M * N$ if the loss is summed (NOT averaged as usual) across instances in a batch (because the gradients between different nodes are averaged). You should take this into consideration when you want to obtain a mathematically equivalent training process compared to the local training counterpart. But in most cases, you can just treat a `DistributedDataParallel` wrapped model, a `DataParallel` wrapped model and an ordinary model on a single GPU as the same (E.g. using the same learning rate for equivalent batch size).

Parameters are never broadcast between processes. The module performs an all-reduce step on gradients and assumes that they will be modified by the optimizer in all processes in the same way. Buffers (e.g. `BatchNorm` stats) are broadcast from the module in process of rank 0, to all other replicas in the system in every iteration.

If you are using `DistributedDataParallel` in conjunction with the Distributed RPC Framework, you should always use `torch.distributed.autograd.backward()` to compute gradients and `torch.distributed.optim.DistributedOptimizer` for optimizing parameters.

`DistributedDataParallel` currently offers limited support for gradient checkpointing with `torch.utils.checkpoint()`. If the checkpoint is done with `use_reentrant=False` (recommended), DDP will work as expected without any limitations. If, however, the checkpoint is done with `use_reentrant=True` (the default), DDP will work as expected when there are no unused parameters in the model and each layer is checkpointed at most once (make sure you are not passing `find_unused_parameters=True` to DDP). We currently do not support the case where a layer is checkpointed multiple times, or when there unused parameters in the checkpointed model.

To let a non-DDP model load a state dict from a DDP model, `consume_prefix_in_state_dict_if_present()` needs to be applied to strip the prefix “module.” in the DDP state dict before loading.

Constructor, forward method, and differentiation of the output (or a function of the output of this module) are distributed synchronization points. Take that into account in case different processes might be executing different code.

This module assumes all parameters are registered in the model by the time it is created. No parameters should be added nor removed later. Same applies to buffers.

This module assumes all parameters are registered in the model of each distributed processes are in the same order. The module itself will conduct gradient allreduce following the reverse order of the registered parameters of the model. In other words, it is users’ responsibility to ensure that each distributed process has the exact same model and thus the exact same parameter registration order.

This module allows parameters with non-rowmajor-contiguous strides. For example, your model may contain some parameters whose `torch.memory_format` is `torch.contiguous_format` and others whose format is `torch.channels_last`. However, corresponding parameters in different processes must have the same strides.

This module doesn’t work with `torch.autograd.grad()` (i.e. it will only work if gradients are to be accumulated in `.grad` attributes of parameters).

If you plan on using this module with a nccl backend or a gloo backend (that uses Infiniband), together with a `DataLoader` that uses multiple workers, please change the multiprocessing start method to `forkserver` (Python 3 only) or `spawn`. Unfortunately Gloo (that uses Infiniband) and NCCL2 are not fork safe, and you will likely experience deadlocks if you don’t change this setting.

You should never try to change your model’s parameters after wrapping up your model

with `DistributedDataParallel`. Because, when wrapping up your model with `DistributedDataParallel`, the constructor of `DistributedDataParallel` will register the additional gradient reduction functions on all the parameters of the model itself at the time of construction. If you change the model's parameters afterwards, gradient reduction functions no longer match the correct set of parameters.

Using `DistributedDataParallel` in conjunction with the Distributed RPC Framework is experimental and subject to change.

`module (Module)` – module to be parallelized

`device_ids` (list of int or `torch.device`) –

CUDA devices. 1) For single-device modules, `device_ids` can contain exactly one device id, which represents the only CUDA device where the input module corresponding to this process resides. Alternatively, `device_ids` can also be `None`. 2) For multi-device modules and CPU modules, `device_ids` must be `None`.

When `device_ids` is `None` for both cases, both the input data for the forward pass and the actual module must be placed on the correct device. (default: `None`)

`output_device` (int or `torch.device`) – Device location of output for single-device CUDA modules. For multi-device modules and CPU modules, it must be `None`, and the module itself dictates the output location. (default: `device_ids[0]` for single-device modules)

`broadcast_buffers` (bool) – Flag that enables syncing (broadcasting) buffers of the module at beginning of the forward function. (default: `True`)

`init_sync` (bool) – Whether to sync during initialization to verify param shapes and broadcast parameters and buffers. WARNING: if this is set to `False` the user is required to ensure themselves that the weights are the same on all ranks. (default: `True`)

`process_group` – The process group to be used for distributed data all-reduction. If `None`, the default process group, which is created by `torch.distributed.init_process_group()`, will be used. (default: `None`)

`bucket_cap_mb` – `DistributedDataParallel` will bucket parameters into multiple buckets so that gradient reduction of each bucket can potentially overlap with backward computation. `bucket_cap_mb` controls the bucket size in MebiBytes (MiB). If `None`, a default size of 25 MiB will be used. (default: `None`)

`find_unused_parameters` (bool) – Traverse the autograd graph from all tensors contained in the return value of the wrapped module's forward function. Parameters that don't receive gradients as part of this graph are preemptively marked as being ready to be reduced. In addition, parameters that may have been used in the wrapped module's forward function but were not part of loss computation and thus would also not receive gradients are preemptively marked as ready to be reduced. (default: `False`)

`check_reduction` – This argument is deprecated.

`gradient_as_bucket_view` (bool) – When set to `True`, gradients will be views pointing to different offsets of allreduce communication buckets. This can reduce peak memory usage, where the saved memory size will be equal to the total gradients size. Moreover, it avoids the overhead of copying between gradients and allreduce communication buckets. When gradients are views, `detach_()` cannot be called on the gradients. If hitting such errors, please fix it by referring to the `zero_grad()` function in `torch/optim/optimizer.py` as a solution. Note that gradients will be views after first iteration, so the peak memory saving should be checked after first iteration.

`static_graph` (bool) –

When set to `True`, DDP knows the trained graph is static. Static graph means 1) The set of used and unused parameters will not change during the whole training loop; in this

case, it does not matter whether users set `find_unused_parameters = True` or not. 2) How the graph is trained will not change during the whole training loop (meaning there is no control flow depending on iterations). When `static_graph` is set to be `True`, DDP will support cases that can not be supported in the past: 1) Reentrant backwards. 2) Activation checkpointing multiple times. 3) Activation checkpointing when model has unused parameters. 4) There are model parameters that are outside of forward function. 5) Potentially improve performance when there are unused parameters, as DDP will not search graph in each iteration to detect unused parameters when `static_graph` is set to be `True`. To check whether you can set `static_graph` to be `True`, one way is to check ddp logging data at the end of your previous model training, if `ddp_logging_data.get("can_set_static_graph") == True`, mostly you can set `static_graph = True` as well.

`delay_all_reduce_named_params` (list of tuple of str and `torch.nn.Parameter`) – a list of named parameters whose all reduce will be delayed when the gradient of the parameter specified in `param_to_hook_all_reduce` is ready. Other arguments of DDP do not apply to named params specified in this argument as these named params will be ignored by DDP reducer.

`param_to_hook_all_reduce` (`torch.nn.Parameter`) – a parameter to hook delayed all reduce of parameters specified in `delay_all_reduce_named_params`.

`skip_all_reduce_unused_params` – When set to `True`, DDP will skip reducing unused parameters. This requires that unused parameters remain the same across all ranks throughout the entire training process. If this condition is not met, it may cause desynchronization and result in training hang.

`module` (`Module`) – the module to be parallelized.

Context manager for training with uneven inputs across processes in DDP.

This context manager will keep track of already-joined DDP processes, and “shadow” the for